

Lists

A list holds values of one type, in order.

```
use std.io

fn main() {
    var scores = [7, 3, 9, 1]
    scores.push(5)
    io.say("{scores}")
    io.say("{scores.len()} items")
    io.say("first {scores.first() ?? 0}, last {scores.last() ?? 0}")
    io.say("item 2 is {scores[2]}")
}
```

Shaping a list

`map`, `filter` and `reduce` make a new list rather than changing the old one.

```
use std.io

fn main() {
    let numbers = [1, 2, 3, 4, 5, 6]
    io.say("{numbers.map(fn(n) => n * n)}")
    io.say("{numbers.filter(fn(n) => n % 2 == 0)}")
    io.say("{numbers.reduce(0, fn(total, n) => total + n)}")
}
```

Asking questions

```
use std.io

fn main() {
    let words = ["apple", "fig", "cherry"]
    io.say("{words.any(fn(w) => w.len() > 5)}")
    io.say("{words.all(fn(w) => w.len() > 2)}")
    io.say("{words.count(fn(w) => w.contains(\"e\"))}")
    io.say("{words.find(fn(w) => w.starts_with(\"f\")) ?? \"none\"}")
}
```

Sorting

`sort` works on anything orderable; `sort_by` takes a comparison that answers "does a come before b".

```
use std.io

fn main() {
    let names = ["Grace", "ada", "Alan"]
    io.say("{names.sort()}")
}
```

```
    io.say("{names.sort_by(fn(a, b) => a.lower() < b.lower())}")
}
```

Sorting is stable, so equal items keep the order you gave them.

Slices

```
use std::io;

fn main() {
    let row = [10, 20, 30, 40, 50]
    io.say("{row.take(2)}")
    io.say("{row.drop(3)}")
    io.say("{row.slice(1, 4)}")
    io.say("{row.reverse()}")
}
```

Exercise

Given a list of numbers, produce a list of running totals: `[1, 2, 3]` becomes `[1, 3, 6]` .